

Data Science Report — LoRA Fine-Tuning on TinyLlama

1. Objective

Develop a lightweight evaluator model that can score interview answers (0–10) and generate short feedback, using LoRA fine-tuning on TinyLlama.

2. Data Preparation

- **Dataset file:** sample_rubric_pairs.jsonl
- **Entries:** ~50 structured QA-feedback examples.
- **Fields:**
 - question — Interview question
 - answer — Candidate’s response
 - score — Numerical evaluation (0–10)
 - feedback — Short qualitative feedback

Example record:

“Explain the difference between precision and recall.”

Score: **9** | Feedback: *Good definitions and formulas; could add an example.*

The dataset includes balanced coverage of 5 core topics: metrics, overfitting, activation functions, missing values, and bias-variance trade-off.

3. Training Overview

- **Training script:** train_lora_tinyllama.py
- **Base model:** TinyLlama (2048 token context)
- **Adapter type:** LoRA
- **Tokenizer:** LlamaTokenizer with BOS = <s>, EOS = </s>
- **Batch size:** 2
- **Epochs:** 1
- **Max steps:** 25
- **Logging interval:** 10 steps

Training Loss Trend:

Step	Epoch	Loss	Grad Norm
------	-------	------	-----------

10	0.4	2.41	3.48
----	-----	------	------

Step Epoch Loss Grad Norm

20 0.8 2.27 2.21

♦ *The loss decreased steadily, suggesting the model learned meaningful patterns even on a small dataset.*

4. Evaluation Summary

(from tiny_llama/eval_results.json)

Metric	Value
Eval loss	0.514
Accuracy	0.738
Log-prob (chosen)	-370.13
Log-prob (rejected)	-296.07

Margin (chosen–rejected) 1.009

Runtime 93.6 s for 2000 samples

♦ *Accuracy of ~74% indicates promising alignment between model predictions and reference scores, despite the minimal dataset.*

5. Key Observations

- LoRA fine-tuning reached convergence within a single epoch.
- Adapter training was successful, but “best_metric” remained null due to early stop before evaluation.
- The dataset is repetitive — future work could diversify question types for better generalization.
- The model tokenization scheme is clean and suitable for low-resource inference.

6. Future Steps

1. Expand dataset with domain-specific rubrics (e.g., SQL, Pandas, ML case studies).
2. Introduce validation split for best-model checkpoint tracking.
3. Visualize loss & accuracy curves using matplotlib for richer report metrics.
4. Integrate evaluator output into Streamlit interface (auto-grading block).